

12주차 누가 이 값을 읽었는지 답할 수 있는가 — 피처 거버넌스와 설명 책임

12주차 · 피처 거버넌스

이 장의 수치는 모두 `practice/chapter12/` 의 코드를 실행해 얻은 값이다. 예시를 위해 지어낸 숫자는 없다. 인용한 값의 출처 파일은 출력 블록 첫 줄에 적었다.

☐ 7주차에서 피처(feature)를 한곳에 모아 훈련과 서빙이 같은 값을 읽게 만들었음 ☐ 그 저장소를 여러 부서와 여러 모델이 함께 쓰기 시작하면 질문이 바뀜 ☐ "값이 맞는가"에서 "이 값은 누가 정의했고, 누가 읽어도 되고, 누가 실제로 읽었는가"로 넘어감 ☐ 이 장은 그 세 질문에 기록으로 답하는 장치를 다룸 ☐ 장치의 이름은 카탈로그·접근 정책·사용 이력·변경 이력임 ☐ 넷 다 사고를 막는 장치가 아니라 **사고 뒤에 경위를 재구성할 수 있게 하는 장치**이며, 그중 하나만 사고를 막음 ☐ 기록이 필요한 이유의 끝에 설명 책임이 있음 ☐ 기관이 자동으로 내린 판정에 시민이 이의를 제기하면 무엇을 근거로 그렇게 판정했는지 답해야 함 ☐ 답할 수 있는지는 **결정이 내려지기 전에 무엇을 적어 두었는지가** 정함 ☐ 이 장도 설명과 실습을 나누지 않음 ☐ 절마다 실행할 명령이 나오고 그 아래에 실제 출력과 해석이 이어짐 ☐ 순서대로 실행하면 이 장의 수치를 직접 생성함

1. 오늘의 질문

☐ 피처의 정의와 소유자를 아무도 적어 두지 않으면 구체적으로 무엇이 깨지는가? ☐ 접근을 **막는 일**과 접근을 **적는 일**은 왜 둘 다 있어야 하는가? ☐ 산식을 고쳤는데 이름을 그대로 두면 누가 무엇을 잘못 읽게 되는가? ☐ "전체 오차 1.0"인 모델이 특정 지역에는 불리할 수 있는가? 그것을 어떻게 보이는가?

2. 이 장에서 실행할 것

☐ 이 장의 장치는 기관 기록관의 열람 절차와 항목이 하나씩 대응함

기록관	이 장의 피쳐 거버넌스
무엇이 보관돼 있고 어느 과가 관리하는지 적은 소장 목록	피쳐 카탈로그 — 정의·소유자·소비자(3절)
직급과 열람 목적에 따라 볼 수 있는 문서가 다른 열람 규정	역할과 행위 단위 접근 정책(4절)
누가 언제 무엇을 왜 열람했는지 적는 열람대장	접근 로그(4절)
문서를 고치면 원본을 두고 개정판 번호를 새로 매김	정의 변경을 새 버전으로 발행(5절)
발급한 증명서에 이의가 들어오면 대장을 꺼내 근거를 설명	자동 판정의 설명과 이의제기(6절)

표 12.1 기록관 열람 절차와 피쳐 거버넌스의 대응

□ 비유는 여기까지다 ○ 기록관은 사람이 열람 신청서를 한 건씩 읽고 판단하므로, 규정에 없는 상황이 오면 담당자가 그 자리에서 물어보고 정할 수 있음 ○ 피쳐는 훈련·서빙 파이프라인이 사람 개입 없이 읽으므로 규칙을 코드로 미리 못 박아 두지 않으면 판정 자체가 일어나지 않음 ○ 그래서 이 장의 정책은 "모르면 물어본다"가 아니라 **"등록돼 있지 않으면 거부한다"** 로 적힘

□ 실행하기 전에 네 가지를 먼저 적음. 남이 만든 산출물을 먼저 보면 거기 적힌 값이 정답처럼 보여 "왜 이 값인가"를 묻지 않게 됨

표 12.2 예측표 — 실행하기 전에 채운다

예측할 것	내 예측	실제	어긋났다면 원인
접근 요청 12건 중 거부는 몇 건이고, 서로 다른 거부 사유는 몇 종류인가			
차단을 빼고 기록만 남기는 설계로 바꾸면 새로 나가는 원시 값 조회는 몇 건인가			
매핑 실패율 상한을 5%로 두면 사흘 중 며칠이 차단되는가			
입력과 무관하게 항상 같은 값을 예측하는 모델의 지역별 오차는 서로 같은가 다른가			

□ 이 장에서 실행할 코드는 셋이며 순서가 정해져 있음

순서	실행할 파일	하는 일	절	선행 조건
1	12-3-catalog-access.py	카탈로그 등록과 접근 요청 12건 판정	3-4	없음
2	12-4-quality-change.py	품질 규칙·임계값·정의 변경·감사 점검표	5	없음
3	12-5-group-bias.py	그룹별 편향·설명·이의제기 통로	6	scikit-learn·pandas·numpy
선택	run_chapter12.py	위 계산을 모두 돌려 증거 JSON을 다시 생성	5-6	3번과 같음

□ 1·2번은 표준 라이브러리만 쓰므로 설치할 것이 없고, 3번만 패키지 세 개가 필요함(실습 검증 환경은 Python 3.13)

```
cd practice/chapter12
pip install -r code/requirements.txt
```

3. 카탈로그 — 피처에 이름표를 붙인다

□ 피처 카탈로그(feature catalog)는 피처마다 **정의·소유자·목적·소비자**를 적어 둔 목록임 ○ 정의는 어떤 산식으로 값을 만드는지 밝힘 ○ 소유자는 그 정의와 품질에 책임지는 부서를 정함 ○ 소비자는 그 값을 읽어 가는 모델과 서비스의 목록임

□ 카탈로그가 없을 때 깨지는 것은 추상적인 "관리 부실"이 아니라 구체적인 두 장면임 ○ **정의를 고칠 때** — 산식을 바꾸려는 사람이 누구에게 알려야 하는지 모름 – 담당자의 기억으로 "아마 훈련 쪽만 쓸 것"이라고 판단하면, 목록에 없던 서빙 쪽이 소리 없이 다른 값을 읽게 됨 ○ **피처를 지울 때** — 아무도 안 쓰는 것 같아 지웠는데 어딘가에서 읽고 있었음 – 소비자가 남아 있는 피처는 폐기하지 않는다는 규칙은 목록이 있어야 지킬 수 있음

□ 카탈로그에 무엇을 적을지는 취향이 아니라 **나중에 누가 무엇을 묻는가가 정함** ○ "이 값이 정확히 무엇을 센 수인가" → 정의 ○ "값이 이상한데 누구에게 말해야 하는가" → 소유자 ○ "이 값을 우리 업무에 써도 되는가" → 목적 ○ "내가 이걸 고치면 누가 영향을 받는가" → 소비자 ○ 네 질문에 답할 항목만 적고, 답할 사람이 없는 항목은 적어도 곧 낚음

□ 카탈로그는 손으로 적는 문서가 아니라 **등록 파일에서 만들어지는 산출물**이어야 함 ○ 손으로 적으면 실제 등록 내용과 어긋나는 순간부터 목록이 거짓말을 하기 시작함 ○ 이 실습의 카탈로그는 7주차에 남긴 등록 파일을 읽어 그대로 만듦

□ 첫 번째 코드를 실행함. 이 절과 4절의 숫자는 전부 이 한 번의 실행에서 나옴

```
cd practice/chapter12
```

```
python code/12-3-catalog-access.py
```

```
# 출처: practice/chapter12/results/12-3-catalog-access.log ([1] 부분)
```

```
[1] 카탈로그 - 정의를 손으로 적지 않고 7주차 등록 파일에서 가져온다
```

```
정의 원천 data/input/ch7_feature_definitions.json (7주차 등록 추출)
```

```
피처 수 2개
```

```
- complaint_count(Int64)
```

```
정의 전일 확정 민원 건수(7장 일별 집계)
```

```
소유자 민원데이터팀(가상 - 시뮬레이션 표기)
```

```
개인정보 개인정보 없음(지역 단위 집계)
```

```
소비자 3곳 - complaint_model_v1 훈련(9장), champion 서빙(10장), 통합 MVP(13주차)
```

```
보존기간 3일, 등록 서비스 complaint_model_v1
```

```
- day_unmapped_rate(Float64)
```

```
정의 전일 정제 품질 - 매핑 실패율(7장 quality_check)
```

```
소유자 민원데이터팀(가상 - 시뮬레이션 표기)
```

```
개인정보 개인정보 없음(지역 단위 집계)
```

```
소비자 1곳 - complaint_model_v1 서비스(9장 훈련)
```

```
보존기간 3일, 등록 서비스 complaint_model_v1
```

□ 화면에 나온 값에서 읽을 것 ○ **정의가 등록 파일에서 왔음** — 두 피처의 정의 문장은 이 실습이 지은 것이 아니라 7주차가 등록해 둔 `ch7_feature_definitions.json` 에서 그대로 옮겨 온 것임. 카탈로그와 실제 등록이 어긋날 여지가 없음 ○ **소비자 수가 3곳과 1곳으로 다름** — `complaint_count` 를 고치면 세 군데에 알려야 하고 `day_unmapped_rate` 를 고치면 한 군데면 됨. 변경의 파급 범위가 숫자로 적혀 있음 ○ **개인정보 등급이 비어 있지 않음** — 두 피처 모두 "개인정보 없음(지역 단위 집계)"임. 등급이 빈칸인 피처는 안전한 것이 아니라 **아직 판정되지 않은 것**이며, 이 차이를 목록이 구분해 줌 ○ **보존기간이 3일임** — 언제까지 두는지도 정의의 일부임. 기한 없이 쌓인 값은 나중에 "왜 아직 갖고 있는가"에 답할 근거가 없음

- **소유자 이름이 "(가상)"인 것은 실습이라서다**
- 실제 기관에서는 부서명과 담당 직위를 적는다. 개인 이름을 적으면 인사이동마다 목록이 낡는다.
- 중요한 것은 이름 자체가 아니라 **묻고 답할 상대가 정해져 있다**는 사실이다.

□ 공공에서 특히 값이 큰 항목은 개인정보 등급임 ○ 등급을 피처 단위로 미리 판정해 두면, 목적 밖 이용 요청이 들어왔을 때 무엇을 근거로 거절할지가 이미 정해져 있음 ○ 등급이 빈칸인 피처를 "개인정보 없음"과 같은 칸에 두면 **판정한 적 없는 값과 판정해서 없다고 결론 낸 값**이 구분되지 않음 ○ 실습의 두 피처는 지역 단위 집계라 개인을 특정하지 못하지만, 가구나 개인 단위로 내려가는 순간 등급도 근거 법령도 다시 따져야 함

□ 카탈로그는 그 자체로 끝나지 않고 4절의 전제가 됨 ○ 목록에 없는 피처는 "권한이 없는" 것이 아니라 **판정의 대상조차 되지 않음** ○ 등록을 건너뛴 값이 조용히 유통되는 경로를 막는 자리가 여기임

4. 인가와 사용 이력 — 하나는 막고 하나는 설명한다

□ 접근 통제에는 성격이 다른 두 가지가 있음 ○ **인가(authorization)** 는 요청이 처리되기 **전에** 허용할지 말지를 정함 ○ **사용 이력(access log)** 은 요청이 처리된 **뒤에** 누가·언제·무엇을·왜 읽었는지를 남김 ○ 둘 중 하나만 두면 각각 다른 방식으로 실패함 – 인가만 두면 사고가 안 나는 대신, 사고가 났을 때 누가 무엇을 봤는지 답하지 못함 – 기록만 두면 경위는 남는 대신, 막을 수 있었던 조회를 그대로 통과시킴

□ 이 실습의 판정 엔진은 세 관문을 순서대로 통과시키고, 하나라도 걸리면 거부함

```
if role not in ACCESS_POLICY:          decision = "DENY"    # 1번 관문
elif feature not in catalog_names:      decision = "DENY"    # 2번 관문
elif action not in ACCESS_POLICY[role]: decision = "DENY"    # 3번 관문
else:                                  decision = "ALLOW"
```

전체 코드는 [practice/chapter12/code/12-1-feature-governance.py](#) 참고

□ 세 관문의 순서에는 뜻이 있음 ○ 1번은 **기본 거부임**. 정책에 이름이 없는 역할은 "아직 안 정했다"가 아니라 "안 된다"로 판정함 ○ 2번은 **카탈로그 게이트**임. 3절의 목록에 없는 피처는 접근 판정 이전에 존재 자체가 인정되지 않음 ○ 3번은 **행위 분리**임. 같은 피처라도 정의를 보는 것(`read_definition`)·값을 읽는 것(`read_value`)·열람 기록을 보는 것(`read_log`)은 다른 권한임 ○ 셋을 한 덩어리로 묶으면 거부 사유가 "권한 없음" 하나로 뭉개지고, 무엇을 고쳐야 하는지가 사라짐

□ 같은 실행의 나머지 출력을 봄

출처: practice/chapter12/results/12-3-catalog-access.log ([2]·[3] 부분, 허용 8건 중 4건 생략)

[2] 접근 요청 - 같은 판정 코드에 12건을 순서대로 넣는다

seq	역할	피쳐	행위	판정	사유
0	training_pipeline	complaint_count	read_value	ALLOW	정책 허용
4	analyst	complaint_count	read_definition	ALLOW	정책 허용
5	analyst	complaint_count	read_value	ALLOW	정책 허용
6	analyst	complaint_count	read_log	DENY	로그 열람은 감사 권한 필요
8	external_auditor	complaint_count	read_log	ALLOW	정책 허용
9	external_auditor	complaint_count	read_value	DENY	감사 역할은 원시 값 조회 불가 -
메타데이터·로그만(목적 제한)					
10	marketing_bot	complaint_count	read_value	DENY	미등록 역할 - 정책에 없음
11	analyst	resident_arrears	read_value	DENY	카탈로그 미등록 피쳐 - 접근의
전제는 등록					
요청 12건 - 허용 8 / 거부 4					

[3] 거부 4건이 걸린 관문 - 서로 다른 지점에서 막혔다

analyst.complaint_count.read_log

3번 관문: 역할에 그 행위 권한 없음

요청 목적: 동료 조회 이력 확인

external_auditor.complaint_count.read_value

3번 관문: 역할에 그 행위 권한 없음

요청 목적: 원시 값 확인 시도

marketing_bot.complaint_count.read_value

1번 관문: 역할이 정책에 없음

요청 목적: 미등록 역할 조회

analyst.resident_arrears.read_value

2번 관문: 피쳐가 카탈로그에 없음

요청 목적: 가구 단위 민감 피쳐 조회

□ 화면에 나온 값에서 읽을 것 ○ **12건 중 8건 허용·4건 거부임** — 중요한 것은 4라는 수가 아니라, 네 건이 서로 다른 이유로 걸렸다는 사실임. 한 관문만 있었다면 나머지는 통과했음 ○ **analyst 가 같은 피쳐에 대해 두 번은 허용되고 한 번은 거부됨** — seq 4·5는 통과하고 seq 6은 막힘. 역할 단위가 아니라 행위 단위로 판정하기 때문임. "분석가는 신뢰할 만하다"로 묶으면 동료의 열람 이력까지 열림 ○ **외부 감사에게 로그는 열어 주고 값은 막음** — seq 8은 허용, seq 9는 거부임. 감사의 목적은 "기관이 규정대로 운영했는가"를 확인하는 것이므로 원시 값이 필요 없음. 권한은 신분이 아니라 **목적에 필요한 만큼**으로 정함 ○ **resident_arrears 는 존재하지도 않는 피쳐임** — 체납액 같은 가구 단위 값을 요청했지만 3절의 카탈로그에 없으므로 2번 관문에서 걸림. 등록을 거치지 않은 피쳐가 몰래 쓰이는 경로를 막는 자리임

□ 그렇다면 "기록만 남기고 막지는 않는" 설계는 어디까지 부족한가

출처: practice/chapter12/results/12-3-catalog-access.log ([4]·[5] 부분)

[4] 같은 요청을 '기록만 하고 막지는 않는' 설계에 넣으면

로그에 남는 줄 수 12줄 - 차단 설계와 같음

실제로 값이 나가는 건수 8건 → 12건

새로 나가는 원시 값 조회 3건

external_auditor.complaint_count - 원시 값 확인 시도

marketing_bot.complaint_count - 미등록 역할 조회

analyst.resident_arrears - 가구 단위 민감 피쳐 조회

새로 나가는 로그 열람 1건

analyst.complaint_count - 동료 조회 이력 확인

기록은 사고를 설명할 뿐 사고를 막지 않는다

[5] 로그가 뒤에 답할 수 있는 질문 - 원시 값을 읽은 주체

training_pipeline: 2건 - 일별 예측 모델 훈련, 훈련 데이터 품질 피쳐

serving_api: 1건 - champion 온라인 예측

analyst: 1건 - 월간 민원 리포트

목적이 함께 적혀 있어야 '등록된 목적 안의 사용이었는가'를 사후에 판정할 수 있다

□ 화면에 나온 값에서 읽을 것 ○ **로그 줄 수는 12줄로 똑같음** — 기록만 하는 설계도 감사 자료는 똑같이 남김. 그래서 "우리는 전부 기록하고 있습니다"라는 말만으로는 통제 수준을 알 수 없음 ○ **값이 나간 건수가 8건에서 12건으로 늘어남** — 늘어난 4건 중 3건이 원시 값 조회임. 미등록 역할과 미등록 피쳐까지 값을 받아 감 ○ **가장 곤란한 한 건은 resident_arrears 임** — 카탈로그에 없는, 곧 개인정보 등급조차 판정된 적 없는 가구 단위 값이 나감. 사고가 난 뒤 로그를 열면 "나갔다"는 사실만 확인됨 ○ **목적이 로그의 한 칸을 차지함** — analyst 의 값 조회 1건에는 "월간 민원 리포트"가 붙어 있음. 목적 없이 역할과 시각만 남기면 나중에 "등록된 목적 안의 사용이었는가"를 판정할 재료가 없음

- 로그를 남기는 것 자체가 새 위험을 만든다
- 접근 로그는 누가 무엇에 관심을 가졌는지를 담은 또 하나의 자료다. 그래서 이 실습은 로그 열람을 별도 권한(read_log)으로 떼어 놓았다.
- 감사 기록에 접근 통제가 없으면, 통제를 확인하려고 만든 기록이 통제 밖에 놓인다.

5. 품질 규칙과 정의 변경 — 값은 조용히 바뀐다

□ 4절까지가 "누가 읽는가"였다면 이 절은 "무엇을 읽는가"임 ○ 접근이 완벽히 통제돼도 읽어 가는 값 자체가 틀렸거나 뜻이 바뀌었으면 통제의 의미가 없음 ○ 여기서 성격이 다른 두 가지를 구분해야 함 - **품질 위반**은 값이 계약을 어긴 것임. 결측·범위 이탈·중복 키는 해석할 대상이 아니라 차단할 대상임 -

드리프트(drift) 는 값의 분포가 시간에 따라 옮겨 간 것임. 계절 변화처럼 정상 원인일 수 있어 조사한 뒤 판정함 ○ 둘을 한 규칙으로 다루면 정상 변화를 막아 세우거나 명백한 오류를 통과시킴(드리프트 감시는 11주차)

□ 두 번째 코드를 실행함

```
python code/12-4-quality-change.py
```

```
# 출처: practice/chapter12/results/12-4-quality-change.log ([2]·[3] 부분)
```

```
[2] 품질 규칙 - 값 하나하나가 계약을 지켰는지 검사한다
```

```
[PASS] 완전성 완전성(결측 없음)
```

```
non-null 9/9
```

```
[PASS] 정확성 정확성(범위: count≥0, rate∈[0,1])
```

```
in-range 9/9
```

```
[PASS] 일관성 일관성(키 유일성)
```

```
unique-keys 9/9
```

```
3중 전부 통과: True
```

```
[3] 임계값은 관례가 아니라 하루 업무에서 역산한다
```

```
(판정 규칙: 그날의 매핑 실패율 > 상한 이면 그날 피처를 차단)
```

```
상한 0.06 차단 0일 없음
```

```
상한 0.05 차단 1일 2026-07-02
```

```
상한 0.045 차단 3일 2026-07-01, 2026-07-02, 2026-07-03
```

```
같은 데이터인데 상한 한 자리를 바꾸면 차단 일수가 0일과 3일 사이를 오간다
```

□ 화면에 나온 값에서 읽을 것 ○ **검사 대상이 9행임** — 6주차가 확정된 사흘치 집계를 3지역으로 나눈 값임. 규칙은 표본이 아니라 전 행에 적용되고, 9/9 처럼 **분모가 함께 적힘**. 분모 없이 "통과"만 적으면 몇 행을 봤는지 알 수 없음 ○ **세 규칙이 서로 다른 것을 봄** — 완전성은 값이 있는지, 정확성은 값이 있을 수 있는 범위인지, 일관성은 같은 키가 두 번 들어왔는지를 봄. 한 규칙으로 뭉뚱그리면 어느 쪽이 깨졌는지 못 밝힘 ○ **상한 0.06과 0.045 사이에서 결과가 0일과 3일로 갈림** — 같은 데이터인데 판정이 완전히 뒤집힘. 그래서 임계값은 "보통 5% 쓴다"로 정할 수 없고, **이 값을 받아 쓰는 업무가 견딜 수 있는 오차에서 거꾸로 계산해야 함** ○ **0.05로 두면 7월 2일 하루가 걸림** — 뒤의 [4] 표를 보면 그날만 물리 18건 중 1건이 빠져 유효 건수가 17건임. 분모가 작아져 같은 실패 1건의 비율이 다른 날보다 높아짐

□ 이제 값이 아니라 **정의**가 바뀌는 경우를 봄 ○ `day_unmapped_rate` 는 "매핑 실패 건수 ÷ 유효 건수"로 정의돼 있었음(v1) ○ 이것을 "매핑 실패 건수 ÷ 물리 건수"로 바꾸자는 제안이 들어옴(v2). 중복까지 포함한 원자료를 분모로 삼자는 뜻임 ○ 둘 다 말이 되는 정의이고, 어느 쪽이 옳은지는 이 절의 논점이 아님. 논점은 **바꾼 뒤에 무슨 일이 일어나는가**임

출처: practice/chapter12/results/12-4-quality-change.log ([4]·[5] 부분)

[4] 정의 변경 - day_unmapped_rate

v1 미매핑 건수 / 유효 건수(valid)

v2 미매핑 건수 / 물리 건수(physical)

날짜	unmapped	valid	physical	v1	v2	값이 바뀌는가
2026-07-01	1	20	20	0.05	0.05	아니오
2026-07-02	1	17	18	0.0588	0.0556	예
2026-07-03	1	22	22	0.0455	0.0455	아니오

값이 바뀌는 날 1일, 영향 행 3행 (그날의 3지역)

영향 소비자 1곳 - complaint_model_v1 서비스(9장 훈련)

요청자 품질관리팀(가상) / 승인자 민원데이터팀 소유자(가상)

[5] 버전 없이 덮어쓰면 - 같은 이름이 두 값을 가리킨다

2026-07-02 day_unmapped_rate

v1 정의로 저장된 값 0.0588

v2 정의로 저장된 값 0.0556

차이 0.0032

이름이 같으므로 소비자는 정의가 바뀐 사실을 모른 채 다른 값을 읽는다

과거 예측을 재현하려면 그때 쓰인 정의가 어느 쪽인지 기록이 있어야 한다

□ 화면에 나온 값에서 읽을 것 ○ **사흘 중 하루만 값이 바뀐** — 7월 1일과 3일은 중복이 없어 유효 건수와 물리 건수가 같으므로 두 정의가 같은 값을 낸. 정의를 바꿔도 대부분의 날에는 아무 일도 안 일어나는 것이 오히려 위험함. 테스트에서 잡히지 않고 넘어감 ○ **바뀐 차이가 0.0032임** — 값이 무너지지도 않고 범위를 벗어나지도 않음. 4절의 세 품질 규칙을 **전부 통과함**. 품질 검사로는 정의 변경을 잡을 수 없음 ○ **영향 범위가 1일 3행·소비자 1곳으로 세어짐** — 3절의 소비자 목록이 없으면 이 "1곳"을 셀 수 없고, 알려야 할 상대를 기억으로 정하게 됨 ○ **요청자와 승인자가 따로 적힘** — 고치자고 한 사람과 책임지는 사람이 같으면 변경 이력이 자기 결재가 됨

□ 그래서 변경은 덮어쓰지 않고 **새 버전으로 발행함** ○ 과거 산출물이 v1을 썼다는 사실이 남아 있어야 그때의 예측을 다시 만들 수 있음(6주차의 재현성이 정의 수준으로 올라온 것) ○ 소비자에게는 "값이 바뀐다"가 아니라 "정의가 바뀌며 당신 쪽은 이 날짜 3행이 달라진다"로 알림

□ 지금까지의 기록은 감사 점검표를 자동으로 채움

출처: practice/chapter12/results/12-4-quality-change.log ([6] 부분, 10항목 중 5개 발췌)

[6] 감사 점검표 - 위 기록에서 자동 생성 (10/10)

1. [PASS] 모든 접근 요청이 로그에 기록되었는가
 4. [PASS] 미등록 역할의 접근이 전부 거부되었는가
 5. [PASS] 감사 역할의 원시 값 조회가 0건인가(목적 제한)
 8. [PASS] 품질 규칙(완전성·정확성·일관성) 3종이 전부 통과했는가
 10. [PASS] 정의 변경이 영향 소비자를 식별했는가
- 사람의 주장이 아니라 카탈로그·로그·품질·변경 이력이 근거다

□ 화면에 나온 값에서 읽을 것 ○ **10항목이 모두 통과함** — 다만 이 표는 사람이 체크한 것이 아니라 3·4·5절의 기록을 읽어 코드가 판정한 것임 ○ **5번 항목은 "0건인가"를 묻고 있음** — 감사 역할이 원시 값을 읽은 적이 0건임을 로그에서 세어 답함. 담당자가 "그런 일 없습니다"라고 답하는 것과 근거가 다름 ○ **자동 점검표는 기록이 있는 만큼만 판정할 수 있음** — 목적을 안 적었다면 목적에 관한 항목을 만들 수 없음. 무엇을 점검할지 정하는 일과 무엇을 기록할지 정하는 일이 사실 같은 일임

6. 설명 책임 — 전체 오차 하나로는 답이 안 된다

□ 공공 기관이 자동 판정을 운영할 때 정확도만으로는 부족함 ○ 시민이나 담당 부서가 "왜 이 결과가 나왔는가"를 물으면 어떤 데이터와 기준이 결과를 만들었는지 답해야 함 ○ 개인정보 보호법 제37조의2는 **중대한 영향을 주는 자동화된 결정**에 대해 거부·설명 요구의 근거를 둬. 조문 대조는 이 과목의 범위 밖이지만, 답할 수 있는 상태를 미리 만들어 두는 일은 이 과목의 범위임

□ 답해야 할 것이 두 가지임 ○ **누구에게 불리한가** — 전체 성능이 아니라 집단별 성능을 나눠 봐야 알 수 있음 ○ **무엇이 결과를 만들었는가** — 모델이 어떤 입력을 실제로 쓰고 있는지 보여야 함

□ 전체 지표 하나가 집단 차이를 감추는 방식은 단순함 ○ 전체 오차는 집단별 오차를 **평균한 값**이므로, 한쪽이 크고 다른 쪽이 작으면 가운데 값이 나옴 ○ 그 가운데 값만 보고서에 적으면 큰 쪽도 작은 쪽도 보이지 않음 ○ 그래서 집단을 나눌 축을 **먼저 정해 두고** 성능을 그 축으로 쪼개 봐야 함. 사고가 난 뒤에 축을 고르면 유리한 축을 고르게 됨

□ 세 번째 코드를 실행함. 9주차가 남긴 훈련 쌍을 그대로 다시 읽어 그때의 배포 모델을 재구성함

python code/12-5-group-bias.py

```
# 출처: practice/chapter12/results/12-5-group-bias.log ([1]·[2]·[3] 부분)

[1] 입력 - 9주차가 남긴 훈련 쌍을 그대로 다시 읽는다
    원천 data/input/ch9_experiment_report.json
    훈련 쌍 6개 (3지역 × 2쌍)
    지문 재계산 96f6b6b3ce9dcdd1 / 9주차 기록 96f6b6b3ce9dcdd1 - 일치 True

[2] 배포 모델(champion)의 전체 성능과 그룹별 성능
    이 모델은 입력과 무관하게 항상 6.0을 예측한다
    champion = 상수 예측

지역      n  실측평균  예측평균  MAE      편향(예측-실측)
강남구     2    7.5      6.0      1.5     -1.5
마포구     2    5.5      6.0      0.5     +0.5
관악구     2    5.0      6.0      1.0     +1.0
전체 MAE 1.0 / 그룹 MAE 격차 1.0 / 비율 3.0배
가장 나쁜 그룹 강남구 / 가장 좋은 그룹 마포구

[3] 후보 모델(challenger, 선형)로 바꾸면
    회귀식 예측 = 3.6818 + 0.4091 × 전일 건수
    전체 MAE는 1.0 → 1.0455 로 나빠지고,
    그룹 격차는 1.0 → 1.4091 로 함께 나빠진다
```

- 화면에 나온 값에서 읽을 것 ○ **지문이 9주차 기록과 일치함** — 이 실습이 쓴 훈련 데이터가 9주차의 그 데이터와 같은 바이트임을 해시로 보임. 설명의 첫 단추는 "무엇을 학습했는가"를 증명하는 것이며, 그 증명이 없으면 뒤의 해석이 전부 주장이 됨 ○ **전체 MAE가 1.0인데 지역별로는 1.5·0.5·1.0임** — 가장 나쁜 지역이 가장 좋은 지역의 3.0배임. 전체 수치 하나만 보고했다면 이 격차는 보이지 않음 ○ **강남구의 편향만 음수임(-1.5)** — 편향은 예측에서 실측을 뺀 값이므로 음수는 **낮춰 잡았다**는 뜻임. 민원이 많은 지역을 계속 적게 예측함 ○ **틀린 방향이 한쪽으로 쏠린 것이 문제임** — 오차가 크더라도 방향이 섞여 있으면 평균적으로는 상쇄됨. 한 지역만 계속 낮게 잡히면, 이 예측으로 인력이나 예산을 배분할 때 그 지역이 매번 덜 받음 ○ **모델을 바꿔도 격차가 줄지 않음** — 선형 모델로 바꾸면 전체 MAE도 격차도 함께 나빠짐 (1.0→1.0455, 1.0→1.4091). "더 나은 모델을 쓰면 해결된다"가 자동으로 성립하지 않음
- 그러면 왜 강남구만 낮춰 잡는가. 모델이 입력을 쓰고 있는지 보면 답이 나옴

```
# 출처: practice/chapter12/results/12-5-group-bias.log ([4] 부분)

[4] 왜 그런가 - 예측이 피처를 쓰고 있는지 본다
  검사 피처 x_prev_count(전일 민원 건수)
  champion    DummyRegressor(mean) - 상수 예측
    permutation importance 0.0
  challenger  LinearRegression - 선형
    permutation importance -0.0455
    계수 slope 0.4091 / intercept 3.6818
  상수 모델의 중요도가 정확히 0 - 값을 섞어도 오차가 그대로다(피처를 안 본다)
```

□ 화면에 나온 값에서 읽을 것 ○ **순열 중요도(permutation importance)** 는 그 입력 열의 값을 뒤섞은 뒤 오차가 얼마나 나빠지는지로 재는 값임. 나빠지면 그 입력을 쓰고 있었다는 뜻이고, 그대로면 안 쓰고 있었다는 뜻임 ○ **배포 모델의 중요도가 정확히 0임** — 전일 건수를 아무리 뒤섞어도 예측이 6.0으로 고정이라 오차가 꿈쩍도 안 함. 이 모델은 지역도 전일 건수도 보지 않음 ○ **그래서 앞의 편향은 우연이 아니라 구조임** — 모든 지역에 같은 값을 내놓으면, 평균보다 민원이 많은 지역은 반드시 과소예측되고 적은 지역은 과대예측됨. 설명이 편향의 원인까지 짚어 줌 ○ **선형 모델의 계수는 그 자체가 설명임** — 전일 건수가 1건 늘면 예측이 0.4091건 늘어남. 다만 이 표본에서는 순열 중요도가 음수라 전일 건수의 기여가 약함

□ 이 결과를 시민에게 답할 수 있는 형태로 바꾸는 것이 마지막 단계임

```
# 출처: practice/chapter12/results/12-5-group-bias.log ([5]·[6] 부분, 검토표 12항목 중 4개 발췌)
```

[5] 이의제기에 답하려면 갖춰야 하는 통로 5개

1. 고지: 완전 자동화된 결정의 기준·절차·처리 방식을 정보주체가 확인하도록 공개
2. 거부권: 권리·의무에 중대한 영향을 주는 결정을 거부할 수 있게 한다
3. 설명요구권: 결정에 대해 설명 등을 요구할 수 있게 한다
4. 인적 개입: 거부·설명 요구 시 정당한 사유가 없으면 인적 재처리·설명 등 조치
5. 기록: 요구·조치를 로그로 남겨 감사에 답한다(12주차 접근 로그 평행)

[6] 공공 AI 검토표 - 위 산출에서 자동 판정 (12/12)

3. [PASS] 편향의 방향(부호)이 식별되었는가
 5. [PASS] 배포 모델의 피처 의존성이 설명되었는가(상수=피처 무시 식별)
 11. [PASS] 소표본 한계가 정직하게 표기되었는가
 12. [PASS] 데이터 지문으로 훈련 데이터가 증명 가능한가(9장 교차)
- 모델 카드 9절 / 정책 루브릭 3기준이 같은 실행에서 생성된다

□ 화면에 나온 값에서 읽을 것 ○ **다섯 통로 중 마지막이 기록임** — 요구와 조치를 로그로 남기는 일은 4절의 접근 로그와 구조가 같음. 이의제기 처리 기록도 누가·언제·무엇을·어떻게 조치했는지를 남김 ○ **셋째 통로(설명)의 재료가 앞 절에 다 있음** — 어떤 데이터로 학습했는지는 지문이, 어떤 입력을 썼는지는 순열 중요도가, 누가 그 값을 읽었는지는 접근 로그가 답함. **사전에 기록하지 않았으면 사후에 만들어 낼 수 없음**

○ **11번 항목이 소표본 한계를 묻고 있음** — 훈련 쌍이 6개뿐이므로 이 실습의 격차 3.0배는 통계적 주장이 아니라 **절차의 시연임**. 한계를 적는 것이 점검 항목에 들어가 있음 ○ **모델 카드(model card)가 같은 실행에서 만들어짐** — 용도·평가 그룹·지표·윤리적 고려·한계를 9개 절로 적은 요약 문서임. 손으로 쓰면 실제 성능과 어긋나지만, 실행 결과에서 생성하면 어긋날 수 없음

□ 모델 카드에서 학생이 눈여겨볼 절은 두 개임 ○ **범위 밖(out-of-scope)** — "이 모델을 어디에 쓰면 안 되는가"를 적는 자리임. 성능을 적는 문서는 많지만 쓰지 말아야 할 곳을 적는 문서는 드물고, 오용은 대개 그 빈칸에서 생김 ○ **윤리적 고려** — 6절에서 실측한 편향과 그 원인이 그대로 들어감. 이 절이 실행 결과에서 채워지지 않고 사람의 문장으로 채워지면, 성능이 바뀐 뒤에도 문서는 옛말을 하고 있게 됨

- 지역구는 법으로 정한 보호 속성이 아니다
- 성별·장애·출신 국가처럼 차별 금지가 걸린 속성과 지역구는 법적 지위가 다르다.
- 다만 지역 형평은 공공에서 실재하는 관심사이고, **집단별로 성능을 나눠 보는 방법 자체는 어떤 속성을 쓰든 같다**. 이 실습은 그 방법을 지역구로 연습한다.

7. 핵심 요약

□ **1. 카탈로그는 피처의 정의·소유자·소비자를 적어, 변경과 폐기의 영향 범위를 기억이 아니라 목록에서 세계 한다.** (3절) ○ 두 피처의 정의는 7주차 등록 파일에서 그대로 왔고, 소비자는 각각 3곳과 1곳으로 적혀 있었음 ○ 소비자 수가 곧 정의를 고칠 때 알려야 할 곳의 수이며, 이 수가 없으면 담당자의 기억이 그 자리를 대신함 □ **2. 인가는 사고를 막고 사용 이력은 사고를 설명하며, 둘 중 하나만으로는 통제가 되지 않는다.** (4절) ○ 요청 12건 중 4건이 거부됐고, 넷은 기본 거부·카탈로그 게이트·행위 분리라는 서로 다른 관문에서 걸렸음 ○ 차단율 빼고 기록만 남기면 로그 줄 수는 12줄로 같은데 값이 나가는 건수는 8건에서 12건으로 늘고, 그중에는 등급조차 판정된 적 없는 `resident_arrears` 조회가 들어 있음 □ **3. 품질 규칙은 값이 계약을 어겼는지를 보고, 정의 변경은 그 규칙을 통과하면서 뜻을 바꾼다.** (5절) ○ 9행에 세 규칙이 모두 통과했지만, 같은 데이터에서 상한을 0.06에서 0.045로 내리자 차단 일수가 0일에서 3일로 뒤집혔음 ○ `day_unmapped_rate` 를 v1에서 v2로 바꾸면 7월 2일 3행만 0.0588에서 0.0556으로 바뀌며, 품질 규칙 셋을 전부 통과하므로 버전과 변경 이력이 없으면 잡히지 않음 □ **4. 전체 오차 하나로는 누구에게 불리한지 답할 수 없고, 집단별로 나눠야 보인다.** (6절) ○ 전체 MAE가 1.0인 배포 모델의 지역별 MAE는 1.5·0.5·1.0으로 3.0배 차이였고, 강남구만 편향이 -1.5로 계속 낮춰 잡혔음 ○ 선형 모델로 바꾸자 격차가 1.0에서 1.4091로 오히려 커져, 모델 교체가 형평 문제를 자동으로 풀지 않음을 보였음 □ **5. 설명 책임은 결정 전에 남긴 기록의 양이 정한다.** (6절) ○ 순열 중요도 0.0이 배포 모델이 입력을 전혀 쓰지 않음을 보였고, 그것이 편향의 원인임을 같은 실행이 짚었음 ○ 이의제기의 다섯 통로 중 설명·기록 두 개는 카탈로그·접근 로그·지문이 미리 있어야 성립하므로, 사후에 만들어 낼 수 없음

8. 과제

□ 실습을 순서대로 실행해 만든 산출물 네 개를 LMS에 제출한다

1. 표 12.2 예측표 — 실행 결과와 대조해 채운 최종본. 텍스트든 이미지든 형식은 무방하다.

2. `practice/chapter12/results/12-3-catalog-access.log`

3. `practice/chapter12/results/12-4-quality-change.log`

4. `practice/chapter12/results/12-5-group-bias.log`

□ 이어서 네 가지를 각각 세 문장 이내로 쓴다

1. 12-3 로그의 [3] 에서 거부 네 건을 표로 옮겨 적고(역할·피처·행위·목적·걸린 관문), 네 관문이 각각 무엇을 막고 있는지 한 줄씩 쓴다.

2. 12-3 로그의 [4] 에서 차단 없이 새로 나가는 요청 한 건을 골라, 그 조희가 실제로 일어났을 때 기관이 어떤 곤란을 겪는지 쓴다.

3. 12-4 로그의 [3] 과 [4] 를 함께 보고, 매핑 실패율 상한을 자기가 정한다면 어느 값으로 할지와 그 근거를 쓴다. 근거는 "이 값을 쓰는 업무가 무엇을 못 견디는가"로 쓴다.

4. 12-5 로그의 [2] 에서 지역별 MAE와 편향을 옮겨 적고, 이 예측으로 민원 처리 인력을 배분할 때 어느 지역이 어떻게 불리해지는지 쓴다.

□ 값을 지어내지 않는다 □ 실행이 안 되면 어디서 막혔는지 그대로 쓴다 □ 수업일 포함 7일 이내에 제출한다

9. 더 알아보기

□ 실습을 더 해 보려면 `practice/chapter12/` 에서 다음을 시도함 ○ 12-1-feature-governance.py 의 `ACCESS_REQUESTS` 에 요청을 한 건 더 넣고, 세 관문 중 어디서 걸리는지 12-3 을 다시 돌려 확인함 ○ 12-1-feature-governance.py 의 `ACCESS_POLICY` 에서 `external_auditor` 에 `read_value` 를 넣어 보고, 감사 점검표 5번 항목이 어떻게 바뀌는지 봄 ○ 12-4-quality-change.py 의 `THRESHOLDS` 에 상한을 하나 더 넣어 차단 일수가 어디서 꺾이는지 확인함 ○ `run_chapter12.py` 로 전체를 돌리면 `data/output/` 에 모델 카드와 정책 루브릭까지 문서로 생성됨. 생성된 `ch12_model_card.md` 의 8절(윤리적 고려)이 6절의 실측값과 어떻게 이어지는지 읽어 봄 □ 뒤 주차에서 이어지는 내용 ○ 이 장의 기록 장치를 실제 서비스 하나에 모두 붙여 보는 것은 13주차에서 다룸 ○ 사람 대신 모델이 품질을 채점할 때 생기는 문제는 14주차에서 다룸 □ 원문을 직접 보고 싶다면 다음 두 편이 출발점임 ○ Mitchell et al. (2019) 「Model Cards for Model Reporting」 — 이 실습이 만드는 모델 카드 9절의 원전. <https://arxiv.org/abs/1810.03993> ○ 개인정보보호위원회 — 자동화된 결정에 대한 정보주체의 권리 안내. <https://www.pipc.go.kr/>